Shopify Clone: Technical Documentation

1. Introduction

1.1 Purpose

The purpose of this document is to provide a comprehensive technical overview and guide for the Shopify Clone project. This platform is designed to empower merchants to create, manage, and scale their online stores with ease, while providing customers with a premium shopping experience.

1.2 Scope

This documentation covers the architectural design, technology stack, installation procedures, configuration requirements, and core functional features of the Shopify Clone. It includes details on multi-tenancy, merchant administration, and storefront management.

1.3 Audience

This guide is intended for:

- **Developers**: To understand the codebase and architecture for further development.

- **System Administrators**: For setting up the environment and deploying the application.

- **Stakeholders**: To gain a high-level understanding of the system's capabilities.

---

2. System Overview

2.1 Architecture

The application utilizes a **Multi-Tenant Architecture** with a shared database and shared schema.

- **Subdomain Routing**: Each store is accessed via a unique subdomain (e.g., `store1.localhost:3000`), handled by Next.js Middleware.

- **Separation of Layers**:

- **Platform Layer**: Main landing page for user acquisition.

- **Admin Layer**: Merchant dashboard for managing store data.

- **Storefront Layer**: Dynamic customer-facing pages.

2.2 Technologies Used

- **Frontend**: Next.js 16, React 19, Tailwind CSS 4.

- **Backend**: Next.js App Router (Server Components & Actions).

- **Database**: PostgreSQL (hosted on Supabase).

- **ORM**: Prisma for type-safe data modeling.

- **Styling**: Modern dark-mode aesthetic with glassmorphism and framer-motion animations.

2.3 Dependencies

Key dependencies include:

- `next-auth`: Authentication.

- `lucide-react`: Iconography.

- `zustand`: Client-side state management.

- `stripe`: Payment processing.

- `cloudinary`: Image storage and optimization.

---

3. Installation Guide

3.1 Prerequisites

- Node.js 18.17 or later.

- pnpm (recommended) or npm.

- A PostgreSQL database (e.g., Supabase, Neon).

- A Cloudinary account.

- A Stripe account.

3.2 System Requirements

- **Memory**: 4GB RAM minimum (8GB recommended for development).

- **Disk Space**: ~500MB for the application and dependencies.

- **OS**: Windows, macOS, or Linux.

3.3 Installation Steps

1. **Clone the repository**:

```bash

git clone

cd shopify_clone

```

2. **Install dependencies**:

```bash

pnpm install

```

3. **Set up the database**:

```bash

npx prisma generate

npx prisma db push
```

```

4. **Run the development server**:

```bash

npm run dev

```

---

4. Configuration Guide

4.1 Configuration Parameters

The system is configured primarily through environment variables.

4.2 Environment Setup

Create a `.env` file in the root directory:

```env

DATABASE_URL="postgresql://..."

NEXTAUTH_SECRET="..."

NEXTAUTH_URL="http://localhost:3000"

GOOGLE_CLIENT_ID="..."

GOOGLE_CLIENT_SECRET="..."

NEXT_PUBLIC_CLOUDINARY_CLOUD_NAME="..."

STRIPE_API_KEY="..."

```
```

4.3 External Services Integration

- **Supabase**: Used as the primary relational database.

- **Cloudinary**: Handles all product and brand image uploads.

- **Stripe**: Manages the checkout flow and payment webhooks.

---

5. Usage Guide

5.1 User Interface Overview

The UI is divided into three main sections:

- **Platform Home**: Futuristic landing page for the service.

- **Merchant Dashboard**: Dark-themed, glassmorphic interface with analytics and management tools.

- **Customer Storefront**: Minimalist, clean design focused on product presentation.

5.2 User Authentication

Users can sign up using email/password or OAuth providers (Google, GitHub) via NextAuth.js.

5.3 Core Functionality

- **Store Creation**: Merchants can initialize a store with a custom subdomain.

- **Product Management**: CRUD operations for products with variants and image support.

- **Order Fulfillment**: Track and manage customer orders in real-time.

5.4 Advanced Features

- **Real-time Analytics**: Visualized sales data.

- **Multi-tenant Routing**: Seamless switching between different stores via subdomains.

5.5 Troubleshooting

- **Routing Issues**: Ensure `localhost` is correctly handled in `middleware.ts`.

- **Database Connection**: Verify `DATABASE_URL` and Prisma schema synchronization.

- **Image Uploads**: Check Cloudinary API credentials and upload presets.